PROJECT BASED LEARNING (PBL)
DATA STRUCTURES AND ALGORITHMS
COURSE CODE: 25CS2103E

PROJECT DOCUMENTATION
SMARTFACTORY — INDUSTRIAL PRODUCTION ANALYTICS PLATFORM

Field	Details
Project Title	SmartFactory — Industrial Production Analytics Platform
Use Case	Industrial production analytics using advanced data structures and algorithms
Programme / Year	B.Tech-2nd year
Section	S-1
Academic Year	2026–27
Faculty / Mentor	Dr Maganti Shyamala
Department	CSE-1

Team Members

S.No.	Student Name	Roll No.	Individual Role / Contribution
1	P Mano Vikas Reddy	2500030607	Developed the string-search and document-indexing modules (M1, M2), including machine-identifier and fault-keyword search and repeated-phrase discovery in maintenance reports.
2	C Sai Sarvan	2500030881	Developed the dynamic-programming and network-flow modules (M3, M4), including typo correction, capacity-based material flow, and worker-to-machine matching.
3	T Krishna Mohan	2500090264	Developed the constraint/approximation and randomized/parallel analytics modules (M5, M6); supported test-case preparation and report integration.

1. Updated Problem Statement and Objectives

1.1 Problem Statement

Modern factories generate large volumes of operational data from machines, production lines, inventory systems, quality-control checks, maintenance reports, and sensor streams. Searching this information, finding recurring faults, correcting inconsistent entries, assigning resources, and detecting production bottlenecks can become difficult as data volume and operational constraints increase.

SmartFactory is an industrial production analytics platform that applies advanced data structures and algorithms to these manufacturing tasks. The implemented platform brings together string matching, suffix-based document indexing, dynamic programming, network flow, constraint modelling, approximation, randomized algorithms, and parallel processing. The project demonstrates how these techniques support information retrieval, production planning, resource allocation, and factory analytics.

1.2 Objectives

• Develop a modular platform for machines, production lines, raw materials, orders, quality reports, maintenance reports, and observations.

• Apply KMP, Z-function, Rabin–Karp, and Aho–Corasick to search identifiers, codes, and fault keywords.

• Use suffix arrays, SA-IS, LCP/Kasai, suffix trees, and suffix automata for indexing and repeated-phrase discovery.

• Apply edit-distance algorithms and dynamic programming to data correction and small optimization problems.

• Model material movement and worker-to-machine assignments using network flow and bipartite matching.

• Represent production constraints and conflicts using SAT-related reductions and vertex-cover approximation.

• Demonstrate randomized and parallel methods for ranking, stream sampling, prefix computation, and aggregation.

• Define test cases and complexity measures for evaluating correctness and scalability.

1.3 Scope of the Project

The scope of the project is the implementation of six DSA modules for manufacturing analytics. It covers text search, document similarity, error correction, selected combinatorial optimization, capacity-flow modelling, conflict approximation, and stream aggregation, applied to the available sample manufacturing data. A live connection to factory equipment, a production database, or industrial control systems is outside the current scope.

2. Final System Architecture



Figure 1. System architecture pipeline adapted from the SmartFactory presentation.

2.1 Architecture Explanation

The implemented pipeline, shown in Figure 1, begins with factory data sources that provide machine identifiers, production orders, inventory codes, maintenance reports, quality-control records, and sensor observations. An ingestion layer validates records before they are stored or indexed. The algorithm layer routes each request to the appropriate module: string search for identifiers and fault keywords; suffix structures for document queries; dynamic programming for correction and selected optimization; network flow and matching for capacity and assignment; approximation for conflict coverage; and randomized or parallel algorithms for stream analytics. The analytics layer presents results such as matching fault records, repeated descriptions, resource assignments, possible bottlenecks, and aggregate production statistics.

2.2 Technology Stack

Category	Technology / Tool	Purpose	Version / Environment
Language	Modular source code (modules M1–M6)	Implement the algorithms and modules	Project source code
Data storage	Structured records and report collection	Store machine, order, inventory, and report data	Sample manufacturing records
Algorithms	M1–M6 DSA modules	Search, indexing, optimization, flow, constraints, aggregation	Implemented as modules M1–M6
Data input	CSV / JSON / generated	Import sample manufacturing data	Sample manufacturing data

	sample records		
Visualization	Program output of search results and analytics	Present results to the user	Program output
Testing	Test cases TC-1 to TC-9	Validate correctness of each module	Test-case table (Section 6.2)

3. Module Description

No.	Module	Purpose	Key Algorithms
1	M1 / CO1 — String Algorithms	Search identifiers, codes, and fault terms	KMP, Z-function, Rabin–Karp, Aho–Corasick
2	M2 / CO2 — Suffix Structures	Index reports and find repeated descriptions	Suffix Array, SA-IS, LCP/Kasai, Suffix Tree, Suffix Automaton
3	M3 / CO3 — Dynamic Programming	Correct entries and optimize small combinations	In the dynamic-programming module (M3), Levenshtein, Damerau–Levenshtein, and weighted edit distance compute the cost of converting a mistyped machine identifier or production entry into a valid one, and a candidate correction is suggested. Matrix-chain multiplication selects an efficient parenthesization for matrix operations. Bitmask DP and subset DP evaluate small combinations of machines or activities; their exponential growth restricts them to small inputs.Distance, Matrix-Chain DP, Bitmask/Subsets DP
4	M4 / CO4 — Network Flow	Model material movement and resource assignment	Ford–Fulkerson, Edmonds–Karp, Dinic, Bipartite Matching, Max-Flow Min-Cut
5	M5 / CO5 — Complexity and Approximation	Represent constraints and cover production conflicts	SAT/3-SAT, reductions, CLIQUE, Independent Set, Vertex Cover 2-approximation
6	M6 / CO6 — Randomized and Parallel	Rank orders and aggregate continuous observations	Randomized QuickSort, Miller–Rabin, Reservoir Sampling, Blelloch Scan, Parallel Reduce, Brent's theorem

Module ownership is as recorded in Section 10 (Individual Contribution of Each Team Member).

4. Implementation Details

4.1 Development Approach

The SmartFactory Industrial Analytics Platform was implemented using algorithms from Data Structures and Algorithms to analyze industrial machine data. The system processes the available dataset and applies suitable algorithmic techniques for machine-data searching, report indexing, fault keyword matching, flow optimization, constraint handling, and data aggregation. The implementation is organized into modules (M1–M6) so that each component performs its assigned task and produces results from the input data. Each module has a defined input and output, which makes the code easier to understand, test, debug, and maintain.

4.2 Core Module I — Search and Document Indexing

In the string and indexing modules (M1, M2), KMP and the Z-function perform single-pattern matching in linear time, Rabin–Karp performs hash-based pattern search, and Aho–Corasick builds an automaton to detect multiple configured keywords in one pass. Suffix arrays and SA-IS index report text; Kasai's LCP algorithm finds shared prefixes among suffixes, which reveals repeated maintenance phrases. Suffix trees and suffix automata provide alternative structures for substring queries.

Example: a report containing "Machine M102 has overheating fault" is searched for the machine identifier and fault keywords, and the positions of the matches are returned. Two maintenance logs containing "vibration spike detected at spindle motor" are compared to identify the recurring phrase.

4.3 Core Module II — Dynamic Programming and Resource Optimization

Levenshtein, Damerau–Levenshtein, and weighted edit distance can suggest corrections for mistyped machine identifiers and production entries. Matrix-chain multiplication chooses an

efficient parenthesization for matrix operations. Bitmask DP and subset DP can evaluate small combinations of machines or activities, but their exponential growth restricts practical input sizes.

4.4 Core Module III — Flow, Constraints, and Parallel Analytics

In the flow and constraint modules (M4, M5), a flow graph represents raw-material sources, production stages, machines, and finished-product destinations. Edge capacities model movement or processing limits; maximum flow and minimum cut identify the constrained path (bottleneck). Bipartite matching models eligible worker-to-machine assignments. SAT/3-SAT and related reductions model production constraints, and a Vertex-Cover 2-approximation covers all edges of a production-conflict graph.

In the randomized and parallel module (M6), Randomized QuickSort ranks production orders, Reservoir Sampling maintains a fixed-size sample from a stream of observations, Blelloch Scan computes prefix aggregates, and Parallel Reduce combines large collections of values. Brent's theorem is used to reason about parallel work and depth. Miller–Rabin is included as a probabilistic primality-testing algorithm in the syllabus mapping, although it is not a central manufacturing operation.

5. DSA Concepts Used

DSA Concept	Where used	How used	Expected output
String matching	M1 search module	Find IDs, production codes, and multiple fault keywords	Matching positions and detected keywords
Suffix structures	M2 document indexing	Index reports and compare repeated substrings	Shared phrases and substring-query results
Dynamic programming	M3 correction and optimization	Calculate edit costs and explore small combinations	Correction suggestions / selected optimum
Network flow	M4 material-flow model	Respect edge capacities and calculate maximum flow	Throughput estimate and minimum-cut bottleneck
Bipartite matching	M4 resource assignment	Connect eligible workers and machine slots	Valid assignment
NP-completeness and approximation	M5 constraint module	Model constraints and approximate vertex cover	Conflict representation and covering set
Randomized algorithms	M6 ranking and number testing	Randomized QuickSort and Miller–Rabin	Sorted orders / probabilistic primality result
Streaming and parallel algorithms	M6 analytics	Reservoir Sampling, Scan, Parallel Reduce	Sample and aggregate statistics

6. Testing Methodology and Test Cases

6.1 Testing Approach

Testing was performed to verify the behavior of the implemented SmartFactory modules. Test cases covered the relevant algorithmic functions and input conditions, including boundary cases and integration of module outputs. Actual outputs were compared with expected behavior, and unexpected results were investigated. The test-case table below records the verification status of each test based on the available results.

6.2 Test Cases

ID	Scenario	Input / action	Expected result	Status
TC-1	String search	Search M102 in a maintenance report	Matching location returned	Tested
TC-2	Multi-keyword scan	Report contains overheating and vibration	All configured keywords detected	Tested
TC-3	Repeated phrase	Two logs share a fault description	Common phrase identified	Tested
TC-4	Typo correction	Input "Machne M102"	Candidate correction suggested if threshold permits	Tested
TC-5	Material flow	Capacity-limited production graph	Maximum flow and bottleneck reported	Tested
TC-6	Worker assignment	Workers with machine eligibility constraints	Valid matching returned	Tested

TC-7	Conflict cover	Run 2-approximation on conflict graph	Every conflict edge is covered	Tested
TC-8	Reservoir sampling	Stream has more than k records	Sample size remains k	Tested
TC-9	Parallel reduction	Aggregate synthetic sensor values	Matches sequential reference result	Tested

Status key: "Tested" means the test case was executed on the implemented module and its output was checked against the expected result. "Passed" is reserved for cases whose recorded test output explicitly confirms success; it is not assigned in this table because no recorded output is attached to this report.

7. Debugging Details

Debugging involved reviewing program logic, checking inputs and outputs, identifying unexpected behavior, correcting logic where necessary, and rerunning the relevant test cases. For each module, results were compared with the expected behavior defined in the test-case table (Section 6.2); when an output differed from the expected result, the module's intermediate values and edge cases were examined to find the cause, the logic was corrected, and the related test cases were rerun. The table below summarizes the checks applied to each area. This process helped verify the behavior of individual modules and improve the reliability of the application.

No.	Area checked	Debugging method	Related test cases
1	String search and keyword matching results	Compare matches with a naive search and test edge cases	TC-1, TC-2, TC-3 rerun after any correction
2	Report index and repeated-phrase results	Check repeated-phrase output on known inputs; observe index size against input size	TC-3 rerun after any correction
3	Flow and matching results	Validate edge capacities, residual edges, and flow conservation	TC-5, TC-6 rerun after any correction
4	Parallel aggregate versus sequential result	Use deterministic test input and compare intermediate values	TC-8, TC-9 rerun after any correction

8. Performance and Optimization

No controlled benchmark has been run, so no measured speedup or timing figure is claimed. The complexity table below gives representative theoretical bounds for the implemented algorithms; actual results depend on the implementation, data distribution, and hardware.

Parameter	Algorithm / method	Representative complexity	Observation
Single-pattern search	KMP / Z-function	$O(n + m)$	Suitable for one pattern
Multi-pattern search	Aho–Corasick	$O(P + n + z)$ after construction	Useful for many keywords
Suffix indexing	SA-IS / Kasai LCP	$O(n)$ construction / LCP	Useful for large text collections
Edit distance	Standard Levenshtein DP	$O(nm)$ time	Input lengths affect cost
Network flow	Edmonds–Karp	$O(VE^2)$	Clear general-purpose baseline
Network flow	Dinic	General bound commonly $O(V^2E)$	Often chosen for larger flow graphs
Matching	Hopcroft–Karp	$O(E\sqrt{V})$	For bipartite assignment
Approximation	Vertex-Cover 2-approx.	$O(V + E)$	At most twice optimum for standard unweighted vertex cover
Sorting	Randomized QuickSort	Expected $O(n \log n)$, worst $O(n^2)$	Pivot randomness affects performance
Sampling	Reservoir Sampling	$O(N)$ time, $O(k)$ sample space	For stream length N and sample size k
Parallel aggregation	Scan / Reduce	Typically $O(N)$ work, $O(\log N)$ depth	Depends on parallel model and implementation

9. Final Results

The SmartFactory Industrial Analytics Platform is a completed implementation of an industrial analytics workflow using algorithmic techniques for searching, indexing, optimization, flow allocation, constraint handling, and data aggregation. Module M1 searches identifiers and fault

keywords; M2 indexes reports and finds repeated phrases; M3 corrects entries and solves small optimization problems; M4 models material flow and worker assignment; M5 represents production conflicts and approximates a covering set; and M6 ranks orders and aggregates stream data. The implemented modules were evaluated using test cases TC-1 to TC-9. The project demonstrates how Data Structures and Algorithms can be applied to industrial data-processing problems through modular implementation and systematic testing.

9.1 Achievement of Objectives

Objective	Evidence / result	Status
Search identifiers and fault keywords	Search outputs for KMP, Z, Rabin–Karp, and Aho–Corasick (TC-1, TC-2)	Completed
Index historical reports	Suffix-array construction and repeated-phrase output (TC-3)	Completed
Correct entries and optimize small tasks	Edit-distance and DP outputs (TC-4)	Completed
Model material flow and assignments	Max-flow result and matching output (TC-5, TC-6)	Completed
Represent production constraints	Constraint graph and approximation output (TC-7)	Completed
Aggregate streaming data	Sampling, scan, and reduce outputs (TC-8, TC-9)	Completed

Objective not yet completed: scalability evaluation. Repeatable timing and memory measurements on larger datasets have not been carried out, so this objective is listed separately and is not marked as completed. It is addressed under Future Enhancements (Section 11.2).

10. Individual Contribution of Each Team Member

The table below records the contribution of each team member to the implemented SmartFactory platform, together with the modules concerned.

No.	Student	Roll No.	Specific contribution	Modules
1	P Mano Vikas Reddy	2500030607	Developed the string-search and document-indexing modules (M1, M2), including machine-identifier and fault-keyword search and repeated-phrase discovery in maintenance reports.	M1 String Algorithms; M2 Suffix Structures
2	C Sai Sarvan	2500030881	Developed the dynamic-programming and network-flow modules (M3, M4), including typo correction, capacity-based material flow, and worker-to-machine matching.	M3 Dynamic Programming; M4 Network Flow and Matching
3	T Krishna Mohan	2500090264	Developed the constraint/approximation and randomized/parallel analytics modules (M5, M6); supported test-case preparation and report integration.	M5 Constraints and Approximation; M6 Randomized and Parallel Algorithms; testing/documentation

11. Limitations and Future Enhancements

11.1 Current Limitations

• The current project focuses on the implemented algorithmic modules and the available sample dataset.

• Real-time operation would depend on a reliable data-ingestion pipeline, timestamps, validation, and suitable storage.

• Suffix structures can require significant memory, and bitmask/subset DP becomes impractical as input size grows.

• Flow and matching results depend on accurate capacity and eligibility data.

• Parallel speedups have not been benchmarked; theoretical work/depth bounds do not guarantee a faster implementation.

• Larger datasets, additional test cases, and performance benchmarking are needed to assess behavior under more demanding conditions; the platform is not yet integrated with live factory equipment.

11.2 Future Enhancements

• Improve processing efficiency by refining the existing modules behind a consistent interface and extending the automated tests.

• Extend the sample dataset for machines, orders, inventory, reports, and sensor events to cover larger and more varied cases.

• Expand fault-search and reporting capabilities, including a dashboard for search results, recurring faults, assignments, and bottlenecks.

• Benchmark time and memory use on increasing dataset sizes.

• Add further input validation, database persistence, and integration with real-time industrial monitoring and streaming ingestion.

• Compare approximation results with exact solutions on small graphs where the optimum is computable.

• Add audit logs and access control for use in a production setting.

12. References

[1] Knuth, D. E., Morris, J. H., and Pratt, V. R. “Fast Pattern Matching in Strings.” SIAM Journal on Computing, 1977.

[2] Aho, A. V., and Corasick, M. J. “Efficient String Matching: An Aid to Bibliographic Search.” Communications of the ACM, 1975.

[3] Kasai, T. et al. “Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications,” 2001.

[4] Levenshtein, V. I. “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” 1966.

[5] Edmonds, J., and Karp, R. M. “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems.” Journal of the ACM, 1972.

[6] Dinic, E. A. “Algorithm for Solution of a Problem of Maximum Flow in Networks with Power Estimation,” 1970.

[7] Vazirani, V. V. Approximation Algorithms. Springer, 2001.

[8] Vitter, J. S. “Random Sampling with a Reservoir.” ACM Transactions on Mathematical Software, 1985.

[9] Blelloch, G. E. “Prefix Sums and Their Applications,” 1990.

[10] Brent, R. P. “The Parallel Evaluation of General Arithmetic Expressions.” Journal of the ACM, 1974.